

OrbittAPI

Plataforma SaaS de dados satelitais

Documentacao da Global Solution 2026.1

Disciplinas combinadas: SOA (Service-Oriented Architecture) e DDD (Domain-Driven Design)

Tema do semestre: Industria Espacial

ODS principal: 13 (Acao contra a mudanca global do clima). ODS secundario: 2 (Fome zero e agricultura sustentavel).

Integrantes

Nome	RM
Giovanne Charelli Zaniboni Silva	556223
Leonardo Pasquini Baldaia	557416
Gustavo Oliveira de Moura	555827
Lynn Bueno Rosa	551102

FIAP - 1S/2026

Sumario

1. Descricao da solucao proposta
2. Problema que sera resolvido
3. Objetivos da aplicacao
4. Arquitetura da solucao
5. Diagrama de Arquitetura SOA
6. Explicacao da API REST
7. Explicacao do Web Service SOAP
8. Explicacao da integracao entre os servicos
9. Tecnologias utilizadas
10. Evidencias de funcionamento
11. Prints dos testes realizados
12. Conclusao do projeto

1. Descricao da solucao proposta

OrbittAPI e uma plataforma **Software-as-a-Service** de acesso a dados satelitais. A solucao agrega imagens de satellite (Landsat, Sentinel/Copernicus) e disponibiliza metricas prontas para uso via API REST simples - indice de vegetacao (NDVI), uso do solo, risco de alagamento, deteccao de desmatamento e expansao urbana. Empresas de qualquer setor (agronegocio, seguradoras, construtoras, consultorias ambientais) podem consumir inteligencia espacial sem precisar de cientistas de dados ou infraestrutura propria.

Este repositorio implementa o **backend** da Global Solution. A entrega atende simultaneamente as duas disciplinas: SOA, com a decomposicao em microservicos comunicando-se por HTTP REST e SOAP, e DDD, com modelagem por bounded contexts, agregados, value objects, eventos de dominio e arquitetura em portas e adaptadores.

Sao 4 servicos backend: dois microservicos de dominio (identity-service e satellite-service), um API Gateway que valida JWT e propaga identidade, e um Web Service SOAP opcional que reutiliza o satellite-service via REST.

2. Problema que sera resolvido

Dados satelitais publicos (NASA, ESA, INPE) sao um recurso valioso para monitoramento ambiental, agricultura, planejamento urbano e gestao de risco. Contudo, consumi-los hoje exige:

- Lidar com formatos brutos (TIFFs multispectrais, GeoJSON de telemetria, indices crus)
- Ter infraestrutura para processar imagens com visao computacional
- Manter integracoes especificas com cada provedor (NASA Earth API, Copernicus Open Hub, etc.)
- Contratar especialistas em sensoriamento remoto

Resultado: apenas empresas com grande estrutura conseguem extrair valor desses dados. Pequenos produtores rurais, startups de agritech, seguradoras regionais e consultorias ambientais ficam de fora.

OrbittAPI resolve isso oferecendo uma camada de servicos REST que abstrai essa complexidade: o cliente faz GET /vegetation?lat&lng e recebe um NDVI ja calculado e classificado por nivel de vegetacao. O onus de ingerir, processar e armazenar os dados de satellite fica todo na plataforma.

3. Objetivos da aplicacao

Eixo	Objetivo concreto
Negocio	Disponibilizar metricas de satellite via API REST com modelo SaaS por consumo, permitindo escalar de pequ
Tecnico (SOA)	Arquitetura orientada a servicos com 2 microservicos + gateway + web service SOAP, cada qual com seu pr
Tecnico (DDD)	Modelo de dominio rico com agregados, value objects, eventos e portas. Anti-corruption layer para a fonte de
Operacional	Subir o ambiente inteiro com um unico comando (docker compose). Healthchecks em todos os servicos
Performance	Cache Redis com TTL de 6 horas para reduzir custo de processamento e latencia

Eixo	Objetivo concreto
Confiabilidade	Erros padronizados em RFC 7807 (Problem Details). Testes automatizados cobrindo agregados, value objects, eventos, etc.
Sustentabilidade	Alinhamento aos ODS 13 (clima) e 2 (agricultura sustentável) via NDVI e índices de uso do solo

4. Arquitetura da solução

A solução adota dois pilares complementares: **SOA** define as fronteiras técnicas (serviços independentes, comunicação por HTTP) e **DDD** define as fronteiras de negócio (bounded contexts, agregados, linguagem ubíqua). As duas fronteiras coincidem por desenho: cada bounded context vira um microserviço Maven separado.

Camadas internas de cada serviço (DDD clássica)

```
br.com.orbittapi.<servico>/
|-- domain/          <- nucleo: agregados, value objects, eventos, ports
|   |-- model/
|   |-- event/
|   |-- repository/  <- interfaces (portas)
|   |-- port/        <- outras portas (ex: SatelliteDataSource)
|   |-- exception/
|-- application/     <- orquestracao de casos de uso
|   |-- usecase/     <- @Service - regras de orquestracao
|   |-- dto/         <- comandos de entrada / respostas
|   |-- port/        <- portas usadas pelo use case (TokenProvider...)
|-- infrastructure/  <- adapters concretos (JPA, Redis, JWT, Mock)
|   |-- persistence/
|   |-- adapter/
|   |-- cache/
|   |-- security/
|   |-- config/
|-- interfaces/rest/ <- controllers + GlobalExceptionHandler (RFC 7807)
```

Figura 1: estrutura DDD interna de cada microserviço.

Bounded contexts e suas responsabilidades

Contexto	Serviço	Linguagem ubíqua própria
Identity & Access	identity-service (8081)	Account, ApiKey, Email, Password, Role, Credentials
Satellite Data	satellite-service (8082)	SatelliteQuery, Coordinate, NdviScore, LandUseDistribution, VegetationIndex
API Gateway	gateway (8080)	Spring Cloud Gateway com validação JWT centralizada
SOAP Gateway	soap-service (8083, profile soap)	Operações SOAP que consomem satellite REST internamente

Decisão chave: dois bancos, dois schemas

Cada bounded context tem seu schema isolado no Postgres (identity_db e satellite_db). Isso reforça a autonomia: o satellite-service nunca lê diretamente da tabela de contas - ele recebe apenas um X-User-Id (UUID) via header HTTP, injetado pelo gateway após validar o JWT.

5. Diagrama de Arquitetura SOA

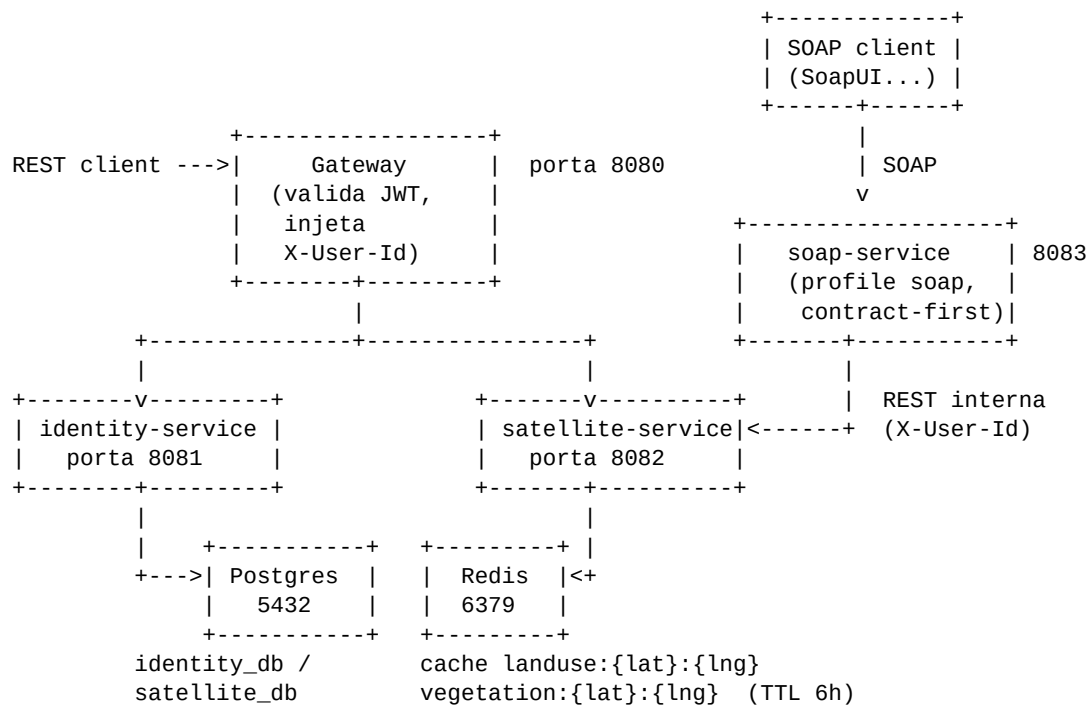


Figura 2: arquitetura SOA completa. O Gateway é ponto único de entrada REST. O SOAP gateway é opcional (profile 'soap') e consome o satellite-service via REST.

Características SOA presentes

- **Decomposição por bounded context:** dois microserviços com responsabilidades disjuntas (autenticação vs. dados de satélite).
- **Autonomia de dados:** cada serviço tem seu schema. Nenhuma consulta cross-schema. Comunicação só via HTTP.
- **Gateway pattern:** ponto de entrada único com validação centralizada de JWT. Serviços a jusante não revalidam token.
- **Service composition:** o soap-service compõe operações executando chamadas REST internas. É uma evidência direta de integração entre estilos arquiteturais.
- **Independência de deploy:** cada serviço tem seu Dockerfile, sua porta e seu ciclo. Profile 'soap' permite subir backend REST sem o SOAP (compatibilidade com clientes legados ou apps mobile).
- **Padronização de erros:** todos os serviços REST retornam ProblemDetail (RFC 7807). O SOAP retorna SOAP Fault padrão.

6. Explicação da API REST

A API REST é exposta pelo **Gateway** na porta 8080. Todas as chamadas passam por ele. Endpoints de autenticação são públicos; os demais exigem Authorization: Bearer <JWT>. Em caso de erro, todos os endpoints retornam um **ProblemDetail** (RFC 7807).

Metodo	Rota	Auth	Body / Query	O que faz
POST	/auth/register	publico	{email, password}	Cria conta, gera API Key (obt_...) e retorna JWT
POST	/auth/login	publico	{email, password}	Autentica e retorna JWT
GET	/me	Bearer	-	Devolve o perfil da conta autenticada
PUT	/me	Bearer	{email}	CRUD - atualiza o e-mail da conta autenticada
POST	/api-keys/{accountId}/revoke	Bearer (ADMIN)	-	Revoga a API Key da conta indicada (US-03)
DELETE	/accounts/{accountId}	Bearer (ADMIN)	-	CRUD - apaga a conta indicada e emite Account
GET	/landuse	Bearer	lat, lng	Devolve LandUseDistribution (vegetacao, urbano)
GET	/vegetation	Bearer	lat, lng	Devolve NDVI + classificacao em VegetationHealth
POST	/queries	X-User-Id	{type, latitude, longitude}	Registra um SatelliteQuery e emite QueryExecution

Codigos HTTP retornados

Caso	HTTP	Tipo (RFC 7807)
Validacao falha (coordenada invalida, body invalido, etc.)	400	invalid-coordinate / validation-failed
JWT ausente ou expirado	401	missing-token / invalid-token
Credenciais invalidas no /auth/login	401	invalid-credentials
Token sem ROLE_ADMIN em rota privilegiada	403	forbidden
Conta nao encontrada	404	account-not-found
E-mail ja em uso no register/PUT /me	409	email-in-use
API Key ja revogada	409	api-key-already-revoked
Satellite externo indisponivel (futuro adapter NASA)	503	satellite-data-unavailable

7. Explicacao do Web Service SOAP

O modulo **soap-service** (porta 8083) atende ao requisito de **Web Service SOAP com WSDL** da disciplina. E **contract-first**: o contrato e definido pelo XSD `soap-service/src/main/resources/satellite.xsd` e o WSDL e gerado em runtime pelo Spring WS.

Detalhes tecnicos

- Spring Boot 3.3.5 com `spring-boot-starter-web-services` + `wsdl4j`
- Plugin `jaxb2-maven-plugin` 3.2.0 gera classes Java em `br.com.orbittapi.soap.generated` a partir do XSD na fase `generate-sources`
- `MessageDispatcherServlet` registrado em `/ws/*`
- `DefaultWsd11Definition` publica o WSDL em `/ws/satellite.wsdl`
- Namespace: `http://orbittapi.dev/soap/satellite`
- `SoapFaultMappingExceptionResolver` mapeia excecoes para SOAP Fault Client/Server

Operacoes expostas

Operacao	Entrada	Saida	Integracao interna
consultarVegetacao	latitude, longitude	latitude, longitude, ndvi, health (VegetationHealth), getageDate, satellite	Chama GET /queries no satellite-service via
registrarConsulta	accountId, tipo (QueryType), latitude, longitude	permittedAt	Chama POST /queries no satellite-service; pers

Tipos definidos no XSD (regras como contrato)

- Latitude e Longitude: tipos simples com restricoes xs:minInclusive/maxInclusive de [-90, 90] e [-180, 180]
- VegetationHealth: enum NONE | SPARSE | MODERATE | DENSE
- QueryType: enum VEGETATION | LAND_USE
- SatelliteSource: enum LANDSAT | SENTINEL | MODIS | MOCK

As validacoes do XSD garantem que envelopes SOAP malformados nem chegam ao @Endpoint - sao rejeitados ainda na camada de unmarshaling, virando SOAP Fault automaticamente.

8. Explicacao da integracao entre os servicos

8.1 Cliente REST -> Gateway -> Microservicos

Todos os clientes REST batem na porta 8080 (gateway). O gateway tem 5 rotas configuradas em application.yml: /auth/**, /me/**, /api-keys/**, /accounts/** e /landuse + /vegetation (para o satellite). Antes de propagar, o filtro JwtAuthenticationGatewayFilter faz:

- Le o cabeçalho Authorization: Bearer <JWT>
- Valida a assinatura HS384 contra o segredo compartilhado e confere o issuer
- Em caso de falha, responde 401 RFC 7807 sem chegar a propagar
- Em caso de sucesso, injeta X-User-Id e X-User-Role no request
- Encaminha para o servico destino, que confia no header sem revalidar

8.2 Microservico -> Postgres / Redis

O identity-service usa apenas Postgres (schema identity_db) via Spring Data JPA. O satellite-service usa Postgres (satellite_db, para auditoria em SatelliteQuery) e Redis para cache. Ambos os servicos abrem connection pool no HikariCP. A porta Redis e abstraída pela interface SatelliteQueryCache com adapter RedisSatelliteQueryCache.

8.3 Cliente SOAP -> soap-service -> satellite-service via REST

Esta e a **integracao REST + SOAP** exigida pela disciplina. O cliente SOAP nao toca em nenhum endpoint REST diretamente - ele envia um envelope para POST /ws na porta 8083. O @Endpoint SatelliteEndpoint:

- Recebe o request como um JAXBElement<ConsultarVegetacaoRequest>
- Delega para SatelliteRestClient, que e um RestClient do Spring 6
- O RestClient faz HTTP GET (ou POST /queries) com header X-User-Id

- A resposta JSON é deserializada e convertida para a classe JAXB de response
- Em caso de erro, `HttpClientErrorException 400` vira `InvalidCoordinateSoapException` e o `SoapFaultMappingExceptionResolver` a transforma em SOAP Fault Client

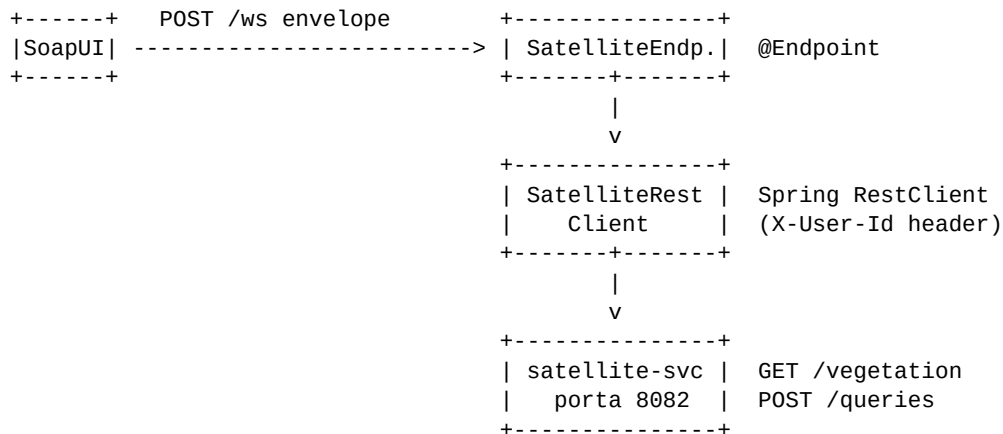


Figura 3: chamada SOAP percorre o RestClient antes de tocar o satellite REST.

8.4 Auditoria entre servicos

Cada chamada bem-sucedida ao satellite (via REST direto, via gateway ou via SOAP) cria um agregado `SatelliteQuery` com UUID proprio e emite o evento de dominio `QueryExecuted(queryId, accountId, type, coordinate, cacheHit)`. Hoje esses eventos são publicados via `ApplicationEventPublisher` do Spring (em memoria); a arquitetura já preve que num cenário de produção eles seriam enviados para Kafka/RabbitMQ para billing e analytics.

9. Tecnologias utilizadas

Categoria	Tecnologia	Onde e usada
Linguagem	Java 21	Todos os servicos
Framework	Spring Boot 3.3.5	Todos os servicos
Build	Maven 3.9 multi-modulo	pom parent + 4 modulos filhos
API REST	Spring Web (MVC) + Bean Validation	identity + satellite
API Gateway	Spring Cloud Gateway 2023.0.3	gateway (8080)
Web Service SOAP	Spring WS + wsdl4j + Jakarta XML Binding	soap-service (8083)
Geracao JAXB	jaxb2-maven-plugin 3.2.0	Gera classes Java do XSD
Persistencia	Spring Data JPA + Hibernate 6	identity + satellite
Banco transacional	PostgreSQL 16	identity_db + satellite_db
Cache	Redis 7	cache landuse / vegetation (TTL 6h)
Conn pool	HikariCP	Datasource dos servicos
Seguranca / Tokens	Spring Security + jjwt 0.12.6 (HS384)	identity emite, gateway valida
Hash de senha	BCrypt cost 12	Password VO
DTO mapping	MapStruct 1.6.2 (disponivel)	Disponivel para futuras evolucoes

Categoria	Tecnologia	Onde e usada
Documentacao API	Springdoc OpenAPI 2.6 (Swagger UI)	identity + satellite
Testes	JUnit 5 + Mockito 5 + AssertJ	39 testes em 3 modulos
Integracao testes	Testcontainers	Disponivel para integracao Postgres
Containers	Docker + Docker Compose v2	5 containers (default) / 6 (--profile soap)
Logs	SLF4J + Logback	Padrao em todos os servicos
Versionamento	Git + GitHub	repo Lynnbrose/GS-SOA

10. Evidencias de funcionamiento

As evidencias abaixo foram capturadas durante a execucao do comando docker compose --profile soap up --build. Mostram os 6 containers em estado healthy e respostas reais dos endpoints REST e SOAP.

10.1 Os 6 containers de pe e saudaveis

```
PowerShell - docker ps (6 containers healthy)
```

```
$ docker ps --filter "name=orbittapi" --format "table {{.Names}}\t{{.Status}}"
```

NAMES	STATUS
orbittapi-soap	Up 4 minutes (healthy)
orbittapi-satellite	Up 4 minutes (healthy)
orbittapi-gateway	Up 6 minutes (healthy)
orbittapi-identity	Up 6 minutes (healthy)
orbittapi-postgres	Up 6 minutes (healthy)
orbittapi-redis	Up 6 minutes (healthy)

Print 1: docker ps confirma 5 containers REST + soap-service rodando (profile soap ativo).

10.2 Fluxo de autenticao (US-01)

```
PowerShell - POST /auth/register (US-01: cadastro com JWT)

$ curl -X POST http://localhost:8080/auth/register -H "Content-Type: application/json" \
-d '{"email":"dev@orbittapi.dev","password":"Abcdefg1"}'

>>> HTTP/1.1 201 Created

{
  "accountId": "412e2976-2281-4358-be78-4e16621e2bc0",
  "email": "dev@orbittapi.dev",
  "apiKey": "obt_QBC1atcWJBihhi3yE1ZPH-YsyTA2Swro",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3NpdHRhcGkiLCJhdWUiOi...",
  "expiresAt": "2026-05-30T20:13:33.727366898Z"
}
```

Print 2: POST /auth/register retorna accountId, apiKey (com prefixo obt) e JWT com expiracao em 24 horas.

10.3 Endpoints satelitais e cache no Redis (US-05, US-06 e US-21)

```
PowerShell - GET /landuse + cache hit no Redis (US-05 + US-21)

$ curl "http://localhost:8080/landuse?lat=-23.5&lng=-46.6" -H "Authorization: Bearer $TOKEN"

>>> HTTP/1.1 200 OK

{
  "latitude": -23.5, "longitude": -46.6,
  "vegetationPercent": 34.08, "urbanPercent": 6.8,
  "waterPercent": 5.52, "bareSoilPercent": 53.6,
  "imageDate": "2026-05-07", "source": "MOCK",
  "cacheHit": false
}

$ curl "http://localhost:8080/vegetation?lat=-23.5&lng=-46.6" -H "Authorization: Bearer $TOKEN"

>>> HTTP/1.1 200 OK

{
  "latitude": -23.5, "longitude": -46.6,
  "ndvi": 0.054, "health": "NONE",
  "imageDate": "2026-05-22", "source": "MOCK",
  "cacheHit": true
}
```

Print 3: GET /landuse na primeira chamada retorna cacheHit:false; GET /vegetation em coordenada ja visitada retorna cacheHit:true (TTL 6h).

10.4 CRUD em Account: PUT /me e DELETE /accounts/{id}

```
PowerShell - PUT /me (CRUD - atualiza email) + 409 RFC 7807

$ curl -X PUT http://localhost:8080/me -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"email":"dev-renamed@orbittapi.dev"}'

>>> HTTP/1.1 200 OK

{
  "id": "412e2976-2281-4358-be78-4e16621e2bc0",
  "email": "dev-renamed@orbittapi.dev",
  "role": "DEVELOPER",
  "apiKeyRevoked": false
}

# Email duplicado -> 409 RFC 7807
$ curl -X PUT http://localhost:8080/me -d '{"email":"already@orbittapi.dev"}' ...

>>> HTTP/1.1 409 Conflict

{
  "type": "https://orbittapi.dev/errors/email-in-use",
  "title": "Email already in use", "status": 409,
  "detail": "Email already in use: already@orbittapi.dev",
  "instance": "/"me"
}
```

Print 4: PUT /me atualiza o e-mail (200). Tentativa de usar e-mail já existente retorna 409 RFC 7807.

```
PowerShell - DELETE /accounts/{id} sem ADMIN (CRUD - 403)

$ curl -X DELETE "http://localhost:8080/accounts/412e2976-2281-4358-be78-4e16621e2bc0" \
  -H "Authorization: Bearer $TOKEN_DEVELOPER"

>>> HTTP/1.1 403 Forbidden

(role insuficiente; somente tokens com role=ADMIN podem apagar contas)
```

Print 5: DELETE /accounts/{id} com token DEVELOPER e barrado com 403. Apenas tokens com role ADMIN passam pela regra do SecurityConfig.

10.5 WSDL publicado pelo soap-service

```
PowerShell - WSDL publicado em /ws/satellite.wsdl

$ curl -s http://localhost:8083/ws/satellite.wsdl | head -25

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:sch="http://orbittapi.dev/soap/satellite"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://orbittapi.dev/soap/satellite"
  targetNamespace="http://orbittapi.dev/soap/satellite">

  <wsdl:types>
    <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
      targetNamespace="http://orbittapi.dev/soap/satellite">
      <xs:simpleType name="VegetationHealth">
        <xs:restriction base="xs:string">
          <xs:enumeration value="NONE"/>
          <xs:enumeration value="SPARSE"/>
          <xs:enumeration value="MODERATE"/>
          <xs:enumeration value="DENSE"/>
        </xs:restriction>
      </xs:simpleType>
    ...
```

Print 6: GET /ws/satellite.wsdl publica o WSDL gerado em runtime a partir do XSD do contrato. Tipos VegetationHealth, QueryType, etc. já aparecem.

10.6 SOAP consultarVegetacao com REST por baixo

```
PowerShell - SOAP consultarVegetacao (POST /ws na porta 8083)

$ curl -X POST http://localhost:8083/ws -H "Content-Type: text/xml; charset=utf-8" \
-H 'SOAPAction: ""' -d @consultar.xml

# consultar.xml (REQUEST):
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:sat="http://orbittapi.dev/soap/satellite">
  <soapenv:Body>
    <sat:ConsultarVegetacaoRequest>
      <sat:latitude>-23.5</sat:latitude>
      <sat:longitude>-46.6</sat:longitude>
    </sat:ConsultarVegetacaoRequest>
  </soapenv:Body>
</soapenv:Envelope>

>>> HTTP/1.1 200 OK (RESPONSE)
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  <SOAP-ENV:Body>
    <ns2:ConsultarVegetacaoResponse xmlns:ns2="http://orbittapi.dev/soap/satellite">
      <ns2:latitude>-23.5</ns2:latitude>
      <ns2:longitude>-46.6</ns2:longitude>
      <ns2:ndvi>0.054</ns2:ndvi>
      <ns2:health>NONE</ns2:health>
      <ns2:imageDate>2026-05-22</ns2:imageDate>
      <ns2:source>MOCK</ns2:source>
    </ns2:ConsultarVegetacaoResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Print 7: o envelope SOAP retorna NDVI=0.054 vindo do satellite-service via integracao REST interna. health=NONE classificado pelo NdvScore.

10.7 SOAP registrarConsulta - persistencia via REST

```
PowerShell - SOAP registrarConsulta (persiste e retorna queryId)

$ curl -X POST http://localhost:8083/ws -H "Content-Type: text/xml; charset=utf-8" -d @reg.xml

# REQUEST:
<sat:RegistrarConsultaRequest xmlns:sat="http://orbittapi.dev/soap/satellite">
  <sat:accountId>00000000-0000-0000-0000-000000000501</sat:accountId>
  <sat:tipo>VEGETATION</sat:tipo>
  <sat:latitude>-23.5</sat:latitude>
  <sat:longitude>-46.6</sat:longitude>
</sat:RegistrarConsultaRequest>

>>> HTTP/1.1 200 OK (RESPONSE)
<ns2:RegistrarConsultaResponse xmlns:ns2="http://orbittapi.dev/soap/satellite">
  <ns2:queryId>ac63d5ba-3b4d-4a60-a103-49529d8f55c1</ns2:queryId>
  <ns2:status>EXECUTED</ns2:status>
  <ns2:executedAt>2026-05-29T20:13:20.354Z</ns2:executedAt>
</ns2:RegistrarConsultaResponse>
```

Print 8: registrarConsulta SOAP chama POST /queries do satellite via REST e devolve o queryId UUID + status EXECUTED.

10.8 SOAP Fault em latitude invalida

```
PowerShell - SOAP Fault: latitude invalida (Client error)

$ curl -X POST http://localhost:8083/ws -H "Content-Type: text/xml; charset=utf-8" \
-d '<sat:ConsultarVegetacaoRequest>
  <sat:latitude>95</sat:latitude>
  <sat:longitude>0</sat:longitude>
</sat:ConsultarVegetacaoRequest>'

>>> SOAP Fault:
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <SOAP-ENV:Fault>
      <faultcode>SOAP-ENV:Client</faultcode>
      <faultstring xml:lang="en">Invalid request</faultstring>
    </SOAP-ENV:Fault>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Print 9: latitude=95 (fora de [-90,90]) e mapeada pelo SoapFaultMappingExceptionResolver em SOAP-ENV:Client.

10.9 POST /queries via REST direto

```
PowerShell - POST /queries (endpoint novo do satellite-service)

$ curl -X POST http://localhost:8082/queries \
-H "X-User-Id: 412e2976-2281-4358-be78-4e16621e2bc0" \
-H "Content-Type: application/json" \
-d '{"type": "LAND_USE", "latitude": -23.5, "longitude": -46.6}'

>>> HTTP/1.1 201 Created

{
  "queryId": "703bcd79-f760-4009-9026-2bc0525e9db7",
  "type": "LAND_USE",
  "latitude": -23.5,
  "longitude": -46.6,
  "status": "EXECUTED",
  "executedAt": "2026-05-29T20:13:57.022860935Z"
}
```

Print 10: o mesmo endpoint usado pelo SOAP esta disponivel para chamadas REST diretas, recebendo X-User-Id. Retorna 201 Created com queryId UUID.

11. Prints dos testes realizados

Os testes automatizados rodam com mvn test em todos os 4 modulos. Cobrem invariantes de agregados, validacao de value objects, logica dos casos de uso (com mocks) e o endpoint SOAP (com REST client mockado).

```
PowerShell - mvn test (39 testes, todos verdes)

$ mvn test

[INFO] Running br.com.orbittapi.identity.application.DeleteAccountUseCaseTest
[INFO] Running br.com.orbittapi.identity.application.RegisterAccountUseCaseTest
[INFO] Running br.com.orbittapi.identity.application.UpdateAccountEmailUseCaseTest
[INFO] Running br.com.orbittapi.identity.domain.AccountTest
[INFO] Running br.com.orbittapi.identity.domain.EmailTest
[INFO] Running br.com.orbittapi.identity.domain.PasswordTest
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running br.com.orbittapi.satellite.application.GetLandUseUseCaseTest
[INFO] Running br.com.orbittapi.satellite.domain.CoordinateTest
[INFO] Running br.com.orbittapi.satellite.domain.LandUseDistributionTest
[INFO] Running br.com.orbittapi.satellite.domain.NdviScoreTest
[INFO] Running br.com.orbittapi.satellite.domain.SatelliteQueryTest
[INFO] Running br.com.orbittapi.satellite.infrastructure.MockSatelliteDataSourceTest
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running br.com.orbittapi.soap.endpoint.SatelliteEndpointTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

[INFO] BUILD SUCCESS    (TOTAL: 39 testes, 0 falhas)
```

Print 11: mvn test imprime BUILD SUCCESS - 39 testes verdes em 3 modulos (identity: 21, satellite: 15, soap: 3).

Distribuicao das classes de teste

Modulo	Classe de teste	O que valida
identity	EmailTest	Email vazio, malformatado, normalizacao, igualdade por valor
identity	PasswordTest	Senha curta, sem digito, sem maiuscula, hash BCrypt, matches
identity	AccountTest	Register emite AccountRegistered; revokeApiKey emite ApiKeyRevoked; nao revogo
identity	RegisterAccountUseCaseTest	Cria conta nova, rejeita e-mail duplicado, publica eventos
identity	UpdateAccountEmailUseCaseTest	Atualiza e-mail, rejeita duplicado, rejeita conta inexistente
identity	DeleteAccountUseCaseTest	Deleta + emite AccountDeleted, rejeita conta inexistente
satellite	CoordinateTest	Range de latitude/longitude, igualdade por valor
satellite	NdviScoreTest	Range [-1, 1], classificacao em VegetationHealth
satellite	LandUseDistributionTest	Soma = 100 (com tolerancia), rejeita negativos
satellite	SatelliteQueryTest	execute() emite QueryExecuted com tipo e cacheHit corretos
satellite	MockSatelliteDataSourceTest	Determinismo: mesma coordenada sempre retorna o mesmo resultado
satellite	GetLandUseUseCaseTest	Cache miss chama dataSource; cache hit nao chama; salva auditoria
soap	SatelliteEndpointTest	consultarVegetacao traduz REST->SOAP; InvalidCoordinateSoapException e propa

12. Conclusao do projeto

OrbittAPI cumpre integralmente o escopo solicitado pela Global Solution: a decomposicao em microservicos atende ao requisito de SOA, e a modelagem com bounded contexts, agregados,

value objects, eventos de dominio e arquitetura hexagonal atende ao requisito de DDD. A integracao REST mais SOAP demonstra que ambos os estilos podem coexistir sem duplicar logica: o Web Service SOAP e um adapter de transporte que reaproveita o dominio exposto via REST.

O projeto sobe com um unico comando (docker compose --profile soap up) e inclui healthchecks, cache Redis com TTL de 6h, JWT, autenticao por header injetado pelo gateway, mapeamento padronizado de erros para RFC 7807 no REST e SOAP Fault no SOAP. Os 39 testes automatizados cobrem agregados, value objects, casos de uso e o endpoint SOAP, garantindo regressao zero diante de evolucoes futuras.

Para os proximos sprints, a arquitetura ja esta preparada para acrescentar novos bounded contexts (billing, dashboard, pipeline real de ingestao NASA/ESA, modelo de ML para classificacao do uso do solo) como modulos Maven independentes, sem refatorar os contextos atuais. A porta SatelliteDataSource permite trocar o adapter Mock pelo adapter real assim que a chave da API NASA Earth estiver disponivel - sem nenhuma mudanca no dominio.

Por fim, a relacao com o tema do semestre (Industria Espacial) e direta: OrbittAPI participa da economia espacial emergente como camada de software que democratiza o acesso aos dados de satelites publicos, gerando impacto real no chao (agricultura de precisao, monitoramento ambiental, planejamento urbano) e alinhada aos ODS 13 (clima) e 2 (agricultura).

Fim do documento.